halue

CAPITOLATO DI PROGETTO

Sonar

Documentazione vivente di uno store Shopify

Ingegneria del Software — A.A. 2026/2027
Corso di Laurea in Informatica
Università degli Studi di Padova

Azienda proponente

Halue S.r.l. Società Benefit

Viale Mentana 41, 43121 Parma (PR) · Via A. Brustolon 14, 35031 Abano Terme (PD)
hello@halue.io · halue.com · P.IVA IT 03028930349

Controllo documentale

Elemento	Dettaglio
Titolo	HAL-IS26-001 — Sonar: documentazione vivente di uno store Shopify — Capitolato di Progetto
Azienda proponente	Halue S.r.l. Società Benefit
Classificazione	Public
Stato rilascio	Released
Versione	1.1.0
Data ultima modifica	15/09/2026
Autore	Valentino Baraldo — Amministratore, Halue S.r.l. SB
Riferimento Halue	Valentino Baraldo — valentino.baraldo@halue.io
Nome file	`HAL-IS26-001 - Sonar - Capitolato di Progetto.md`

Tabella delle revisioni

Rev.	Data	Descrizione	Autore	Verifica	Approvazione
0.1.0	02/09/2026	Prima stesura: contesto, problema, obiettivi e descrizione del sistema	Valentino Baraldo	—	—
0.2.0	07/09/2026	Stesura requisiti, architettura di riferimento e stack tecnologico	Valentino Baraldo	Valentino Baraldo	—
1.0.0	11/09/2026	Integrazione criteri di accettazione, glossario e riferimenti; versione rilasciata	Valentino Baraldo	Valentino Baraldo	Valentino Baraldo
1.1.0	15/09/2026	Recepimento dei rilievi di revisione: acronimi di dominio spiegati e glossario ampliato con riferimenti, spazio di negoziazione sui requisiti obbligatori, presidio dei consumi delle piattaforme, casella di posta dedicata	Valentino Baraldo	Valentino Baraldo	Valentino Baraldo

Indice

1. L'azienda proponente **4**

2. Il problema **4**

3. La soluzione: Sonar **5**

4. Descrizione del sistema **5**

4.1 Acquisizione (scansione) 5

4.2 Normalizzazione e modello a grafo 6

4.3 Analisi del codice di tema 7

4.4 Motore di rilevamento (orfani e codice morto) 7

4.5 Presentazione e documentazione generata 8

5. Architettura di riferimento **8**

6. Tecnologie di riferimento **9**

6.1 Vincolanti 9

6.2 Fortemente consigliate 10

6.3 Flessibilità 11

7. Requisiti **11**

7.1 Requisiti funzionali obbligatori 11

7.2 Requisiti funzionali desiderabili 12

7.3 Requisiti funzionali opzionali 13

7.4 Requisiti di qualità e sicurezza 13

7.5 Requisiti non funzionali 13

8. Fuori ambito (non-obiettivi) **14**

9. Deliverable del progetto **14**

9.1 Analisi dello stato dell'arte 14

9.2 Proof of Concept (PoC) — allineato alla Requirements and Technology Baseline (RTB) 14

9.3 Minimum Viable Product (MVP) — allineato alla Product Baseline (PB) 15

9.4 Sintesi dei deliverable 15

10. Documentazione richiesta **15**

11. Criteri di accettazione **16**

12. Ambiente di lavoro **16**

13. Modalità di collaborazione e supporto **17**

14. Uso dell'intelligenza artificiale nello sviluppo **18**

15. Riferimenti aziendali **19**

Appendice A — Glossario **19**

Appendice B — Riferimenti tecnici **21**

1. L'azienda proponente

Halue è una società benefit italiana che si definisce *Human Value Technology* partner: accompagna aziende ed enti nella trasformazione digitale del commercio, dalla valutazione iniziale all'implementazione e alla gestione continuativa.

Le aree di intervento sono e-commerce B2C (*business-to-consumer*, vendita al consumatore finale) e B2B (*business-to-business*, vendita tra imprese) su **Shopify Plus** e **Salesforce Commerce Cloud**, CRM (*Customer Relationship Management*) su Salesforce, system integration e iPaaS (*integration Platform as a Service*), AI services, UX/UI design (*User Experience* e *User Interface*) e formazione. Halue è **Shopify Partner Plus** nel Service Track e lavora con marchi come Colnago, Dainese, Philippe Model Paris, Piazza Italia, Rossignol, Velasca, Davines e Comfort Zone.

Questo capitolato nasce da un problema concreto e ricorrente: uno **store enterprise** — un negozio online di grandi dimensioni, con molti prodotti, un traffico consistente e più persone che vi hanno lavorato nel corso degli anni — diventa via via più difficile da comprendere e da gestire con sicurezza semplicemente perché esiste da tempo. Oggi Halue risolve questo problema a mano, store per store.

2. Il problema

Uno store enterprise si stratifica nel tempo.

Al lancio è pulito: un tema, quattro o cinque app, un pugno di campi personalizzati, qualche automazione. Dopo tre anni di vita — due cambi di responsabile e-commerce, una campagna di Natale fatta di corsa, un'agenzia subentrata a un'altra, sei app installate per provarle e mai disinstallate — lo stesso store è una sedimentazione di:

- **app installate** di cui nessuno ricorda più il perché, alcune ancora fatturate mensilmente;
- **webhook** (notifiche che Shopify invia automaticamente a un indirizzo esterno quando accade un evento nello store — voce di glossario in Appendice A) registrati verso endpoint di fornitori che non lavorano più con il cliente, e che continuano a essere invocati a ogni ordine;
- **campi personalizzati** (metafield, metaobject) definiti per una funzionalità mai rilasciata, o valorizzati su 12 000 prodotti ma letti da nessuno;
- **script e automazioni** — script tag legacy, Shopify Functions (estensioni che modificano la logica nativa di Shopify — sconti, spedizioni, pagamento — eseguite sull'infrastruttura Shopify; cfr. glossario e shopify.dev/docs/apps/build/functions), workflow di automazione — che agiscono sul comportamento dello store senza che nessuno sappia più chi li ha scritti;
- **codice di tema** con sezioni, snippet e asset non più referenziati da alcun template, che continuano a essere trasportati a ogni deploy.

Il punto non è l'accumulo in sé. È che **questa conoscenza non è scritta da nessuna parte**: vive nella testa delle persone che hanno lavorato allo store. Quando cambia il fornitore, quando il referente interno cambia ruolo, quando passano abbastanza mesi, la conoscenza semplicemente si perde.

Le conseguenze sono quotidiane e concrete:

Situazione	Costo reale
Onboarding di un nuovo team su uno store esistente	2–4 settimane di esplorazione manuale prima di poter toccare qualcosa con sicurezza
Passaggio di consegne tra agenzie	Ricostruzione a mano su fogli di calcolo, sempre incompleta, mai aggiornata

Situazione	Costo reale
Valutazione di un replatforming (migrazione dello store a un'altra piattaforma e-commerce — cfr. glossario)	Impossibile stimare senza sapere cosa c'è davvero e cosa è ancora in uso
Audit di spesa sulle app	Si paga per app che nessuno usa da mesi
Modifica a un campo personalizzato	Nessuno sa dire con certezza quale parte del tema o quale app lo legge: si procede per tentativi
Incidente in produzione	Il tempo di diagnosi si allunga perché manca la mappa delle dipendenze tra le entità reali dello store — quale app, webhook o riga di codice del tema dipende da cosa

L'unica documentazione che regge nel tempo è quella che **non dipende dalla disciplina di chi la scrive**: deve essere ricostruita dal sistema stesso, automaticamente e ripetutamente.

3. La soluzione: Sonar

Sonar è un'applicazione web che si collega a uno store Shopify, ne interroga sistematicamente le configurazioni e il codice, e **ricostruisce la mappa di cosa esiste, di cosa dipende da cosa e di cosa non serve più**.

Le quattro domande a cui Sonar deve saper rispondere, in qualunque momento e senza intervento umano:

1. **Cosa è installato?** — app, script, funzioni, automazioni, canali di vendita, temi.
2. **Cosa ascolta quali eventi?** — quali webhook e quali automazioni reagiscono a quali eventi dello store, e verso quali destinazioni.
3. **Quali campi personalizzati esistono e chi li usa davvero?** — non solo l'elenco delle definizioni, ma il loro tasso di valorizzazione reale e i punti del codice che li leggono.
4. **Cosa è orfano?** — definizioni mai valorizzate, webhook verso endpoint morti, app senza alcun riferimento nello store, sezioni di tema non referenziate, asset non usati.

Il risultato è duplice:

- una **documentazione sempre aggiornata** dello store, consultabile via interfaccia web ed esportabile in un formato portabile (Markdown/HTML), che sopravvive al cambio di fornitore;
- un **rapporto di igiene tecnica** che segnala il codice morto, le configurazioni orfane e le dipendenze fragili, con una motivazione esplicita per ogni segnalazione.

Per Halue è lo strumento che accompagna ogni passaggio di consegne e ogni discovery di replatforming. Per il team di progetto è un esercizio pieno di esplorazione di API reali, di normalizzazione di dati eterogenei e di inferenza su evidenze parziali — cioè esattamente il tipo di problema in cui l'ingegneria del software si vede.

4. Descrizione del sistema

Il sistema si articola in cinque capacità.

4.1 Acquisizione (scansione)

Sonar esegue una **scansione** dello store: una sequenza orchestrata di interrogazioni verso le API di Shopify e verso i file del tema, che produce uno **snapshot** immutabile e datato dello stato dello store.

La scansione è un'operazione **lunga** (minuti, su store con decine di migliaia di prodotti), **soggetta a rate limiting** (limitazione della frequenza di chiamata: le Admin API di Shopify adottano un modello a punti con recupero graduale — cfr. glossario e shopify.dev/docs/api/usage/rate-limits) e **parzialmente fallibile** (una risorsa può non essere accessibile senza che questo debba invalidare l'intera scansione). Deve quindi essere progettata come processo durevole, ripartibile e osservabile, non come una singola richiesta HTTP — i tre concetti sono approfonditi nel glossario (voce *Durabilità*) e, più diffusamente, in vercel.com/docs/workflows/concepts e nell'articolo introduttivo vercel.com/blog/a-new-programming-model-for-durable-execution; la sezione 5 mostra come Vercel Workflows li implementa concretamente per Sonar.

Le famiglie di risorse da acquisire:

Famiglia	Contenuto	Fonte
App	App installate, scope concessi, sviluppatore, URL di lancio, abbonamenti attivi	Admin GraphQL — `appInstallations`
Eventi	Sottoscrizioni webhook, endpoint di destinazione, topic, formato	Admin GraphQL — `webhookSubscriptions`, `eventBridgeWebhookSubscription`…
Automazioni	Workflow di automazione dello store, trigger, condizioni, azioni	Import di export `.flow` dall'admin
Estensioni di logica	Shopify Functions attive (sconti, pagamento, spedizione), script tag legacy	Admin GraphQL — `shopifyFunctions`, `scriptTags`, `discountNodes`
Dati personalizzati	Definizioni metafield per ogni `ownerType`, definizioni metaobject, istanze	Admin GraphQL — `metafieldDefinitions`, `metaobjectDefinitions`
Canali e servizi	Publication e canali di vendita, carrier service, fulfillment service, profili di spedizione	Admin GraphQL — `publications`, `carrierServices`, `fulfillmentServices`
Tema	Elenco temi, file Liquid/JSON/JS/CSS, `settings_data.json` con i blocchi app attivi	Admin GraphQL — `themes`, `OnlineStoreThemeFile`
Evidenze d'uso	Campionamento statistico su prodotti, varianti, collezioni, ordini, clienti per misurare la valorizzazione effettiva dei campi personalizzati	Admin GraphQL — query campionarie / bulk operations

> **Nota tecnica.** Alcune informazioni non sono ottenibili da un'unica fonte: ad esempio, un'app può registrare webhook che non sono visibili ad altre app. Parte del valore del progetto sta proprio nel **triangolare fonti diverse** — API, file di tema, export manuali — e nel dichiarare esplicitamente il livello di confidenza di ogni informazione ricostruita. Il sistema non deve mai presentare come certo ciò che ha inferito.

4.2 Normalizzazione e modello a grafo

I dati acquisiti sono eterogenei per forma e semantica. Sonar li normalizza in un **modello a grafo** unico, dove:

- i **nodi** sono le entità dello store (un'app, un webhook, una definizione di metafield, un file di tema, una funzione, un'automazione, un canale…);
- gli **archi** sono le dipendenze tra entità (*l'app X registra il webhook Y*, *il template Z legge il metafield W*, *la sezione S è referenziata dal template T*, *lo script tag Q è servito dal dominio dell'app X*);
- ogni arco porta con sé la **fonte dell'evidenza** e un **livello di confidenza** (accertato / probabile / ipotizzato).

Il modello a grafo è il cuore del sistema: è ciò che permette di rispondere a "cosa si rompe se tolgo questo" invece che al solo "cosa esiste".

4.3 Analisi del codice di tema

Per collegare le configurazioni al codice che le usa, Sonar analizza staticamente i file del tema:

- **riferimenti a campi personalizzati** nei template Liquid (`product.metafields.<namespace>.<key>`, accessi dinamici, oggetti metaobject);
- **grafo di inclusione** tra template, sezioni e snippet (`{% render %}`, `{% section %}`, `{% sections %}`, riferimenti in `templates/*.json`);
- **riferimenti agli asset** (`asset_url`, import in JavaScript, `@import` in CSS);
- **blocchi app** (app block e app embed) dichiarati in `config/settings_data.json` e nei template JSON, con lo stato di abilitazione;
- **script di terze parti** inseriti a mano nel tema (tag `<script>` verso domini esterni), che costituiscono una categoria di dipendenza invisibile alle API.

L'analisi deve reggere codice reale: temi con centinaia di file, Liquid non sempre ben formato, JavaScript minificato. L'obiettivo non è un parser perfetto ma un **risultato onesto**: ciò che non si riesce ad analizzare va segnalato come tale, mai ignorato in silenzio.

4.4 Motore di rilevamento (orfani e codice morto)

Sul grafo consolidato opera un **motore di regole** che produce le segnalazioni. Ogni regola ha un identificativo, una severità, una motivazione leggibile e le evidenze che l'hanno attivata.

Esempi di regole attese:

ID	Segnalazione	Severità	Evidenza
`R-MF-01`	Definizione di metafield con tasso di valorizzazione 0%	Media	Campionamento su N entità
`R-MF-02`	Metafield valorizzato ma privo di qualunque lettore noto (*dark data*)	Alta	Assenza di archi entranti dal tema e dalle app
`R-WH-01`	Webhook il cui dominio di destinazione appartiene a un'app non più installata	Alta	Correlazione app ↔ dominio
`R-WH-02`	Più webhook sovrapposti sullo stesso topic verso destinazioni diverse	Media	Raggruppamento per topic
`R-APP-01`	App installata priva di qualunque riferimento (nessun webhook, nessun blocco, nessun script)	Media	Nodo isolato nel grafo
`R-APP-02`	App con blocchi presenti nel tema ma tutti disabilitati	Media	`settings_data.json`
`R-THM-01`	Sezione o snippet non referenziato da alcun template	Bassa	Grafo di inclusione
`R-THM-02`	Asset presente nel tema e mai referenziato	Bassa	Grafo di inclusione
`R-SCR-01`	Script tag legacy attivo servito da un dominio sconosciuto	Alta	Correlazione dominio ↔ app
`R-FN-01`	Shopify Function installata ma non collegata ad alcuna regola attiva	Media	Assenza di archi verso sconti/regole

Il motore deve essere **estensibile**: aggiungere una regola non deve richiedere di modificare il codice di scansione o quello di presentazione.

4.5 Presentazione e documentazione generata

L'output è fruibile in tre modi:

1. **Interfaccia web** di navigazione: catalogo delle entità con filtri e ricerca, scheda di dettaglio per entità, visualizzazione del grafo delle dipendenze, cruscotto delle segnalazioni.
2. **Diff tra snapshot**: confronto tra due scansioni con l'elenco di cosa è comparso, scomparso o cambiato — la funzionalità che rende la documentazione *vivente* invece che fotografica.
3. **Esportazione**: generazione di un bundle di documentazione autoconsistente (Markdown e/o HTML, con diagrammi) scaricabile e versionabile nel repository del cliente, pensato per essere consegnato a un fornitore subentrante.

5. Architettura di riferimento

L'architettura **non è imposta** dall'azienda proponente. Quella che segue è l'ipotesi che Halue ritiene coerente con i vincoli tecnologici della sezione 6; il gruppo è libero di proporne una diversa, purché la scelta sia motivata a partire dall'analisi dei requisiti e approvata da Halue.

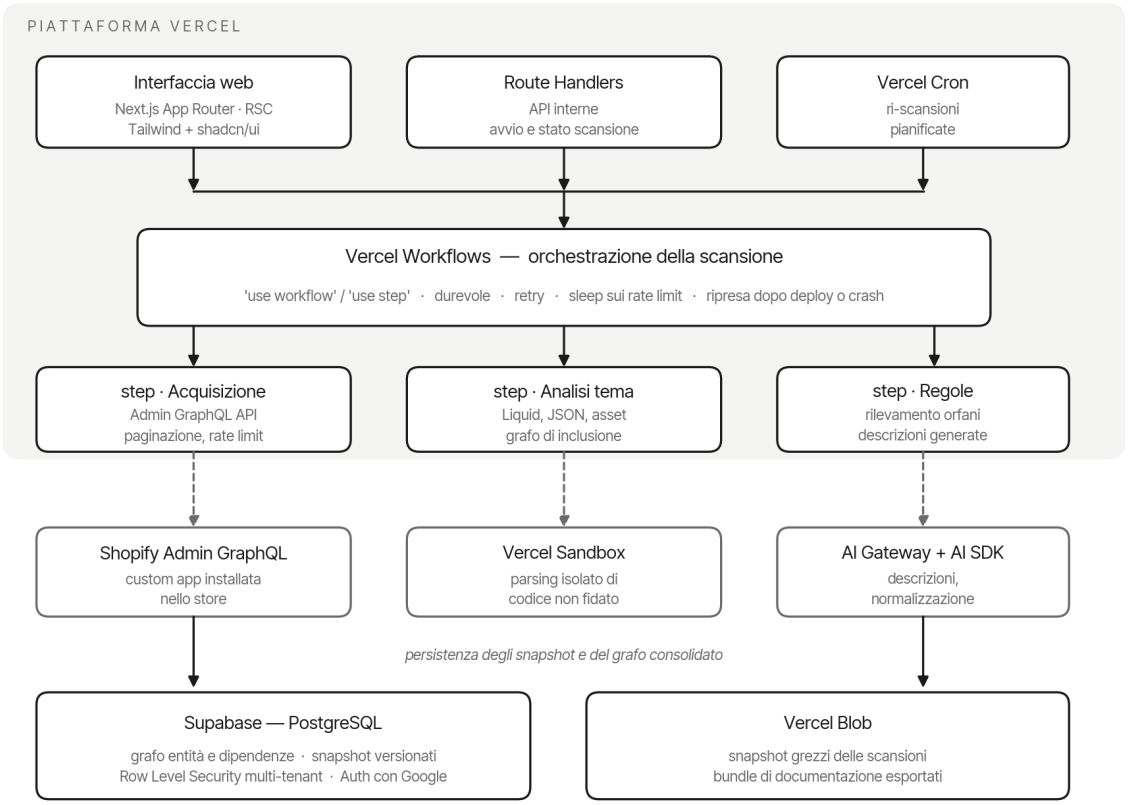


Figura 1 — Architettura di riferimento (ipotesi non vincolante)

Principi architetturali attesi:

- **Separazione netta tra acquisizione, normalizzazione, analisi e presentazione.** Aggiungere una nuova famiglia di risorse Shopify, o una nuova regola di rilevamento, non deve toccare gli altri strati. Questa è la proprietà su cui Halue valuterà più severamente il progetto.
- **Snapshot immutabili.** Una scansione non sovrascrive la precedente: le aggiunge. Il diff è una conseguenza di questa scelta, non una funzionalità a parte.

- **Durabilità dell'orchestrazione.** Una scansione deve poter attraversare un deploy, un errore di rete o un rate limit di venti minuti senza ripartire da zero.
- **Confidenza esplicita.** Ogni informazione derivata porta con sé la propria fonte. Il sistema distingue sempre ciò che ha letto da ciò che ha dedotto.
- **Idempotenza.** Rieseguire una scansione sullo stesso stato deve produrre lo stesso grafo.

6. Tecnologie di riferimento

Halue chiede che il progetto sia realizzato sull'**ecosistema Vercel**, con **Supabase** come base dati relazionale. È lo stack su cui l'azienda sta consolidando i propri strumenti interni, ed è la condizione perché il prodotto finale sia effettivamente adottabile.

6.1 Vincolanti

Ambito	Tecnologia	Motivazione
Framework e hosting	**Next.js** (App Router) su **Vercel**	Unico artefatto per UI, API e job; preview deployment per ogni PR
Orchestrazione	**Vercel Workflows** / Workflow SDK, *Software Development Kit* (`'use workflow'`, `'use step'`)	Scansioni durevoli, riprendibili, osservabili senza gestire infrastruttura
Pianificazione	**Vercel Cron Jobs**	Ri-scansioni periodiche automatiche
Base dati relazionale	**Supabase** (PostgreSQL) con Row Level Security	Modello relazionale per grafo e snapshot, isolamento multi-tenant
Autenticazione	**Supabase Auth — unicamente provider Google (OAuth 2.0 / OIDC,* OpenID Connect)*	Nessuna gestione di password; allineato all'identità aziendale dei clienti Halue
Linguaggio	**TypeScript** (strict)	Coerenza con l'ecosistema e con le pratiche interne
Sorgente dati	**Shopify Admin GraphQL API** tramite *custom app*	Unica via per accedere a `appInstallations` e alle risorse di configurazione
Versionamento	**Git** / GitHub	Repository del gruppo, con integrazione continua (CI, *Continuous Integration*)

6.2 Fortemente consigliate

Ambito	Tecnologia
Archiviazione oggetti	**Vercel Blob** — snapshot grezzi e bundle di documentazione esportati
Coda e ripresa	**Vercel Queues** (sottostante a Workflows) per i lavori a fan-out
Esecuzione isolata	**Vercel Sandbox** — parsing di codice di tema non fidato in ambiente isolato
Livello AI	**Vercel AI Gateway** + **AI SDK** — generazione di descrizioni in linguaggio naturale, normalizzazione dei nomi, raggruppamento semantico. **Mai** per decidere se un'entità è orfana: quella decisione resta deterministica e spiegabile

Ambito	Tecnologia
Accesso alla base dati	**Drizzle ORM** (*Object-Relational Mapping*) o **Prisma**, con migrazioni versionate
Interfaccia	**Tailwind CSS** + **shadcn/ui**; **React Flow** o **Cytoscape.js** per la visualizzazione del grafo
Osservabilità	**Vercel Observability** (traccia dei run di Workflow) + **OpenTelemetry**
Test	**Vitest** (unità e integrazione), **Playwright** (end-to-end), **MSW** (*Mock Service Worker*) per simulare le API Shopify
CI	**GitHub Actions** + Vercel Preview Deployments

6.3 Flessibilità

Le tecnologie della sezione 6.2 sono suggerimenti: alternative sono ammesse purché **motivate per iscritto**. Le tecnologie della sezione 6.1 sono **vincolanti**; una richiesta di deroga va presentata entro la fase di analisi dei requisiti e argomentata.

7. Requisiti

I requisiti sono classificati in **obbligatori** (la base su cui Halue valuta il progetto), **desiderabili** (attesi in un prodotto completo, negoziabili in base all'avanzamento) e **opzionali** (valorizzano il risultato, non sono attesi).

L'elenco degli obbligatori non è però rigido per definizione: se il gruppo incontra vincoli di tempo o di risorse che ne rendono insostenibile la copertura integrale, può proporre a Halue una riduzione motivata dell'ambito — quali requisiti spostare tra i desiderabili, o rimuovere, e perché. La proposta va presentata per iscritto e concordata esplicitamente con Halue, con un margine di preavviso ragionevole rispetto alla verifica finale (non a lavoro già concluso); una volta concordata, ridefinisce l'elenco su cui si applica il criterio di accettazione 1. In assenza di un accordo scritto in tal senso, l'elenco originale resta quello vincolante.

7.1 Requisiti funzionali obbligatori

ID	Requisito
RFO-01	Il sistema deve permettere a un utente autenticato di **registrare uno store Shopify** fornendo le credenziali di una custom app (dominio dello store, token di accesso), verificandone la validità e i permessi disponibili prima del salvataggio.
RFO-02	Il sistema deve permettere di **avviare una scansione** di uno store registrato e di seguirne l'avanzamento in tempo reale (fase corrente, percentuale, risorse acquisite, errori parziali).
RFO-03	La scansione deve essere **durevole**: deve sopravvivere a errori transitori, a rate limiting prolungato e a un nuovo deploy dell'applicazione, riprendendo dall'ultimo passo completato senza ripetere il lavoro già svolto.
RFO-04	Il sistema deve **rispettare i limiti di frequenza** delle Admin API di Shopify, adattando dinamicamente il ritmo delle richieste al credito residuo, senza mai generare errori di throttling non gestiti.
RFO-05	La scansione deve acquisire almeno: **app installate** con i relativi scope, **sottoscrizioni webhook**, **script tag**, **definizioni di metafield** per tutti i principali `ownerType`, **definizioni e istanze di metaobject**, **Shopify Functions**, **canali di vendita/publication**, **elenco dei temi e file del tema pubblicato**.
RFO-06	Il sistema deve **analizzare staticamente il tema pubblicato** e ricostruire: il grafo di inclusione tra template, sezioni e snippet; i riferimenti ai campi personalizzati; i riferimenti agli asset; i blocchi app dichiarati e il loro stato di abilitazione.

ID	Requisito
RFO-07	Il sistema deve **misurare l'uso effettivo** di ogni definizione di metafield mediante campionamento sulle entità proprietarie, producendo un tasso di valorizzazione con la dimensione del campione e l'intervallo di incertezza dichiarati.
RFO-08	Il sistema deve **consolidare i dati in un modello a grafo** unico di entità e dipendenze, in cui ogni arco riporta la fonte dell'evidenza e un livello di confidenza.
RFO-09	Il sistema deve **persistere ogni scansione come snapshot immutabile e datato**, senza sovrascrivere le precedenti.
RFO-10	Il sistema deve applicare un **motore di regole estensibile** che produca segnalazioni di orfani e codice morto; ogni segnalazione deve avere identificativo, severità, spiegazione leggibile e riferimento alle evidenze.
RFO-11	Devono essere implementate **almeno otto regole di rilevamento** distinte, coprendo le famiglie: campi personalizzati, webhook, app, tema, script.
RFO-12	Il sistema deve offrire un'interfaccia di **navigazione del catalogo**: elenco delle entità con ricerca testuale, filtri per tipo, stato e severità, e ordinamento.
RFO-13	Ogni entità deve avere una **scheda di dettaglio** che mostri i propri attributi, le dipendenze entranti e uscenti e le segnalazioni che la riguardano.
RFO-14	Il sistema deve offrire una **visualizzazione grafica delle dipendenze**, navigabile e filtrabile, che a partire da un'entità mostri il suo intorno nel grafo.
RFO-15	Il sistema deve calcolare e mostrare il **diff tra due snapshot** dello stesso store: entità aggiunte, rimosse e modificate, con evidenza delle modifiche significative.
RFO-16	Il sistema deve **esportare un bundle di documentazione** autoconsistente (Markdown e/o HTML, con i diagrammi inclusi) che descriva lo stato dello store e sia consegnabile a terzi.
RFO-17	L'accesso all'applicazione deve avvenire **esclusivamente tramite autenticazione Google**; non devono esistere né registrazione con password né altri provider di identità.
RFO-18	Il sistema deve essere **multi-tenant**: un utente accede esclusivamente agli store della propria organizzazione, con isolamento imposto a livello di base dati (Row Level Security), non solo a livello applicativo.
RFO-19	Il sistema deve **pianificare ri-scansioni automatiche** con cadenza configurabile per store e notificare l'esito.
RFO-20	Il sistema deve **degradare in modo controllato**: se una famiglia di risorse non è accessibile (permessi insufficienti, errore persistente), la scansione prosegue, la lacuna è registrata ed è visibile all'utente come copertura parziale, senza mai essere presentata come assenza di dati.

7.2 Requisiti funzionali desiderabili

ID	Requisito
RFD-01	**Import dei workflow di automazione** dello store da file di export, con integrazione nel grafo di trigger, condizioni e azioni.
RFD-02	**Descrizioni in linguaggio naturale** generate da modello linguistico per le entità prive di documentazione, chiaramente marcate come generate e mai usate per decidere lo stato di un'entità.
RFD-03	**Annotazioni umane**: possibilità di aggiungere note e di marcare manualmente un'entità come "intenzionale, non rimuovere", con conservazione dell'annotazione attraverso le scansioni successive.
RFD-04	**Stima dell'impatto di rimozione**: dato un candidato alla rimozione, elenco delle entità che ne dipendono e valutazione del rischio.

ID	Requisito
RFD-05	**Analisi comparativa di più temi** (pubblicato e non pubblicato), per individuare divergenze e configurazioni rimaste in bozza.
RFD-06	**Notifiche via e-mail** sul completamento delle scansioni pianificate e sulla comparsa di nuove segnalazioni ad alta severità.
RFD-07	**Esportazione in formato macchina** (JSON e JSON-LD, *JSON for Linking Data*) del grafo completo, per l'integrazione con strumenti terzi.

7.3 Requisiti funzionali opzionali

ID	Requisito
RFOp-01	**Applicazione embedded in Shopify Admin** tramite App Bridge.
RFOp-02	**Linea di comando** per avviare una scansione e produrre l'esportazione in ambienti di integrazione continua.
RFOp-03	**Cruscotto di andamento storico**: evoluzione nel tempo del numero di entità, delle segnalazioni aperte e di un indice sintetico di igiene dello store.

7.4 Requisiti di qualità e sicurezza

ID	Requisito
RQS-01	I token di accesso agli store devono essere **cifrati a riposo** e non devono mai comparire in log, messaggi di errore, risposte API o interfaccia utente (se non mascherati).
RQS-02	L'isolamento tra organizzazioni deve essere **imposto dalla base dati** tramite Row Level Security, e verificato da test automatici che ne dimostrino l'efficacia anche in caso di errore applicativo.
RQS-03	Tutte le comunicazioni devono avvenire su **TLS** (*Transport Layer Security*); la gestione dei segreti deve avvenire tramite variabili d'ambiente cifrate, mai in repository.
RQS-04	Il sistema deve applicare il **principio del minimo privilegio** sugli scope Shopify richiesti, documentando per ciascuno la ragione della richiesta.
RQS-05	Il sistema deve mantenere un **registro delle azioni** (chi ha registrato o rimosso uno store, chi ha avviato una scansione, chi ha annotato un'entità).
RQS-06	Il parsing di contenuti non fidati (codice di tema) deve avvenire in modo **isolato e con limiti di risorse**, resistendo a input malformati o ostili senza compromettere il servizio.
RQS-07	Il sistema deve essere **resiliente ai fallimenti parziali**: nessun errore su una singola risorsa deve invalidare l'intera scansione.

7.5 Requisiti non funzionali

ID	Requisito
RNF-01	**Copertura dei test**: copertura di istruzioni ≥ **70%** complessiva, verificata automaticamente in CI.
RNF-02	**Test book**: documento che descriva casi di test, precondizioni, passi, risultati attesi e ottenuti, e criteri di validazione.

ID	Requisito
RNF-03	**Estensibilità**: l'aggiunta di una nuova regola di rilevamento deve richiedere la sola creazione di un nuovo modulo di regola e del relativo test, senza modifiche agli strati di acquisizione, normalizzazione e presentazione. Va dimostrata con un esempio documentato.
RNF-04	**Riproducibilità**: l'ambiente di sviluppo deve essere avviabile con un numero contenuto di comandi documentati, a partire da un file di esempio delle variabili d'ambiente.
RNF-05	**Versionamento**: sviluppo su Git con storia leggibile, convenzioni di commit dichiarate, pull request con revisione tra pari e integrazione continua obbligatoria prima del merge.
RNF-06	**Manutenibilità**: analisi statica (lint, type-check) senza errori in CI; assenza di dipendenze con vulnerabilità note di severità alta o critica al momento della consegna.
RNF-07	**Internazionalizzazione**: interfaccia in **inglese**, con architettura predisposta all'aggiunta di altre lingue.

8. Fuori ambito (non-obiettivi)

Per delimitare con chiarezza il perimetro, **non** rientrano nel progetto:

- **La modifica dello store.** Sonar è uno strumento di sola lettura. Non disinstalla app, non cancella metafield, non modifica temi. Al più **suggerisce** e produce istruzioni da eseguire a mano.
- **Il monitoraggio in tempo reale.** Il sistema fotografa lo stato su richiesta o su pianificazione; non è un sistema di osservabilità continua.
- **L'analisi delle prestazioni dello store** (Core Web Vitals, tempi di risposta del negozio).
- **La gestione dei dati di business** (prodotti, ordini, clienti) come dominio applicativo: le entità di business sono lette solo come evidenza statistica dell'uso dei campi personalizzati.
- **L'addestramento di modelli di machine learning.** L'uso di modelli linguistici è limitato a generazione di testo e normalizzazione, tramite API gestite.
- **Il supporto di piattaforme diverse da Shopify.** Sonar è, per scelta, uno strumento specifico per Shopify: il valore sta nella profondità con cui conosce quella piattaforma, non nella genericità. Non è richiesta né attesa alcuna astrazione della sorgente dati in vista di altri e-commerce; modellare il dominio direttamente sulle entità Shopify è la scelta corretta.

9. Deliverable del progetto

9.1 Analisi dello stato dell'arte

Prima fase, breve: ricognizione degli strumenti esistenti di documentazione automatica e di rilevamento di codice morto, sia in ambito e-commerce sia in ambito generale (analisi statica, *architecture decision records*, cataloghi di servizi). Obiettivo: stabilire cosa esiste già, cosa è riusabile e in cosa Sonar si differenzia.

L'attività è condotta in autonomia dal gruppo: Halue non dispone al momento di materiale proprio da mettere a disposizione, ma è disponibile a discutere i risultati e a dare un riscontro su quanto emerso.

9.2 Proof of Concept (PoC) — allineato alla Requirements and Technology Baseline (RTB)

Prototipo verticale che dimostri la fattibilità dell'**anello critico** del sistema:

- connessione a uno store Shopify di sviluppo tramite custom app;

- scansione di **almeno due** famiglie di risorse (obbligatoriamente app e definizioni di metafield) orchestrata con Vercel Workflows, con gestione dimostrata del rate limiting e ripresa dopo interruzione;
- persistenza su Supabase di un primo modello a grafo;
- analisi di un tema ridotto con estrazione dei riferimenti ai metafield;
- **almeno due** regole di rilevamento funzionanti;
- interfaccia minima con autenticazione Google e visualizzazione dell'elenco delle entità.

Al PoC si accompagnano: analisi dei requisiti con casi d'uso in UML (*Unified Modeling Language*), architettura di massima con diagrammi, piano di qualifica, norme di progetto.

9.3 Minimum Viable Product (MVP) — allineato alla Product Baseline (PB)

Prodotto completo che soddisfi **tutti** i requisiti obbligatori della sezione 7.1 e una quota concordata dei desiderabili, corredato da:

- codice sorgente completo nel repository del gruppo, con storia di sviluppo leggibile;
- applicazione distribuita e funzionante su Vercel, raggiungibile da Halue;
- suite di test automatici con la copertura di RNF-01 verificata in CI;
- test book completo;
- documentazione tecnica e manuale utente;
- esempio di bundle di documentazione esportato dallo store di sviluppo;
- presentazione finale con dimostrazione dal vivo.

9.4 Sintesi dei deliverable

#	Deliverable	Fase
D1	Analisi dello stato dell'arte	Avvio
D2	Analisi dei requisiti con casi d'uso UML	RTB
D3	Specifica architetturale con diagrammi (modello C4 + UML)	RTB
D4	Proof of Concept funzionante	RTB
D5	Piano di qualifica e norme di progetto	RTB
D6	MVP distribuito e funzionante	PB
D7	Suite di test automatici + report di copertura	PB
D8	Test book	PB
D9	Documentazione tecnica e manuale utente	PB
D10	Esempio di documentazione generata dallo store di sviluppo	PB

10. Documentazione richiesta

Per l'intero ciclo di vita è attesa documentazione che comprenda:

- **Diagrammi architetturali** secondo il modello **C4** (contesto, contenitori, componenti) per la vista d'insieme, e **UML** (casi d'uso, classi, sequenza, attività) per il dettaglio. Per grafici di processo e diagrammi leggeri si consiglia **Mermaid**, così da mantenere i diagrammi versionati insieme al codice.
- **Registro delle decisioni architetturali (ADR)**: per ogni scelta significativa — modellazione del grafo, strategia di campionamento, gestione del rate limiting, struttura del motore di regole — un documento breve con contesto, alternative valutate, decisione e conseguenze. Halue considera questo artefatto parte integrante della consegna.
- **Documentazione tecnica** del codice e delle interfacce, chiara e consultabile.
- **Manuale utente** con procedura di configurazione della custom app Shopify, degli scope necessari e dell'uso quotidiano.
- **Test book** completo secondo RNF-02.
- **README operativo** che consenta a una persona esterna al gruppo di avviare il progetto in locale.

11. Criteri di accettazione

Il prodotto è considerato accettato da Halue quando:

1. **Tutti** i requisiti obbligatori (RFO-01 … RFO-20, RQS-01 … RQS-07, RNF-01 … RNF-07) risultano soddisfatti e tracciati nel test book — o, se l'ambito è stato ridotto secondo la procedura della sezione 7, tutti quelli sopravvissuti alla riduzione concordata per iscritto con Halue.
2. La percentuale di casi di test con esito **PASS** è ≥ **95%**, e **nessun** caso fallito riguarda un requisito obbligatorio.
3. La soglia di copertura di RNF-01 è raggiunta e verificata automaticamente in CI.
4. Una scansione completa sullo **store di sviluppo costruito dal gruppo** (sezione 12) si conclude con successo, e la documentazione generata è giudicata da Halue **utilizzabile in un passaggio di consegne reale** — criterio qualitativo, valutato in sede di dimostrazione finale.
5. Il **tasso di falsi positivi** delle segnalazioni ad alta severità, misurato sull'insieme di controllo derivato dallo store di sviluppo, è ≤ **10%**. Ogni falso positivo residuo deve essere spiegabile a partire dalle evidenze mostrate.
6. L'estensibilità di RNF-03 è dimostrata aggiungendo dal vivo, o documentando passo per passo, una regola non prevista.
7. Il repository è consegnato con storia leggibile, istruzioni di avvio funzionanti e nessun segreto in chiaro.

12. Ambiente di lavoro

Il gruppo costruisce autonomamente il proprio ambiente di lavoro. Halue non fornisce store, dati né infrastruttura: tutto ciò che serve è disponibile su piani gratuiti.

Lo store Shopify. Il programma Shopify Partners consente di creare **development store gratuiti e senza scadenza**, con accesso completo alle funzionalità di amministrazione e alle Admin API. Il gruppo ne crea uno e lo popola deliberatamente con la stratificazione che Sonar deve saper riconoscere:

- alcune app installate dall'App Store, preferendo quelle con piano gratuito, di categorie diverse (recensioni, marketing, spedizioni, popup);
- definizioni di metafield con tassi di valorizzazione differenti: alcune popolate su molti prodotti, altre su pochi, altre mai;
- alcuni metafield valorizzati e deliberatamente mai letti dal tema;

- webhook registrati verso destinazioni di comodo, alcune riconducibili ad app poi disinstallate;
- un tema personalizzato con sezioni, snippet e asset volutamente non referenziati da alcun template;
- qualche script tag legacy e qualche script di terze parti inserito a mano nel codice del tema.

Questo lavoro fa parte del progetto, non lo precede. Costruire uno store stratificato obbliga a capire il dominio prima di provare a mapparlo, e produce come sottoprodotto l'**insieme di controllo** con cui misurare i falsi positivi (criterio di accettazione 5): siccome la stratificazione è stata creata dal gruppo, la verità è nota per costruzione. È l'unico modo onesto di valutare un rilevatore di orfani.

Il gruppo documenta la composizione dello store in un artefatto versionato, così che chiunque possa ricostruirlo da zero.

Le piattaforme. Vercel, piano **Hobby** (vercel.com/pricing), e Supabase, piano **Free** (supabase.com/pricing), offrono un livello gratuito sufficiente per lo sviluppo e per la dimostrazione finale — con due limiti concreti di cui il gruppo deve tenere conto per tempo, non scoprirli a ridosso della consegna: il piano Free di Supabase sospende automaticamente un progetto dopo una settimana di inattività, e il piano Hobby di Vercel impone soglie mensili modeste su Workflows, Blob, Sandbox e AI Gateway (quest'ultimo, in particolare, limitato a poche centinaia di richieste al mese) — un vincolo che riguarda direttamente le ri-scansioni pianificate (RFO-19) e la generazione di descrizioni (RFD-02). Il repository Git è del gruppo, su GitHub.

Il presidio dei consumi. Capire per tempo se una scelta tecnica porterà a sforare i limiti gratuiti è difficile all'inizio, quando non si ha ancora idea di quanti eventi generi una scansione o quante richieste consumi la generazione delle descrizioni. Halue non lascia questo punto alla sensibilità del gruppo: è materia degli incontri, con questa scansione.

- **Entro la seconda settimana** il gruppo apre i progetti Vercel e Supabase e annota i limiti del piano gratuito che riguardano Sonar: eventi Workflows, operazioni e spazio Blob, ore Sandbox, richieste AI Gateway, dimensione della base dati, sospensione per inattività. Non serve capirli tutti a fondo, serve averli davanti.
- **A ogni incontro quindicinale** si guardano insieme i consumi effettivi sulle dashboard delle due piattaforme e si stima quanto durerebbero al ritmo corrente. È un punto fisso dell'ordine del giorno, non una voce da sollevare se ci si ricorda.
- **Il consumo va reso misurabile dal prodotto stesso**: ogni scansione registra quante chiamate all'Admin API, quanti step di workflow e quante richieste al modello linguistico ha comportato. Discende dall'osservabilità già richiesta da RFO-02, e serve al gruppo prima ancora che a Halue.
- **Se un limite si avvicina**, la decisione è congiunta e si prende prima di sbatterci contro: ridimensionare la funzionalità, spostarla tra i desiderabili secondo la procedura della sezione 7, oppure far coprire il costo a Halue. Nessuna delle tre penalizza il gruppo, purché il problema emerga all'incontro giusto e non alla consegna.

13. Modalità di collaborazione e supporto

Metodo di lavoro. Halue si aspetta un processo **agile** con iterazioni brevi e incrementi dimostrabili. Il gruppo è autonomo nella scelta della declinazione specifica, purché documentata nelle norme di progetto.

Cadenze.

Attività	Frequenza	Modalità
Incontro di allineamento con il proponente: dimostrazione dell'incremento e revisione dei consumi (sezione 12)	**Ogni due settimane, 45 minuti**	Videoconferenza

Attività	Frequenza	Modalità
Domande, chiarimenti, invio di materiale	Continuativo	Casella di posta dedicata al progetto sonar@halue.io

Canali. La comunicazione asincrona passa da una **casella di posta dedicata al progetto** — non dall'indirizzo personale del referente — presidiata da Halue con risposta entro **un giorno lavorativo**; l'indirizzo viene comunicato al gruppo all'avvio. La scelta è deliberata: una casella dedicata non compete per l'attenzione con la corrispondenza ordinaria di chi la legge, che è il difetto per cui un indirizzo personale sarebbe un canale peggiore. Per la stessa ragione Halue non apre un canale di messaggistica condiviso: richiederebbe un presidio continuo che l'azienda non può garantire con la qualità che il gruppo merita, e un canale presidiato male è peggio di un canale che non c'è. Le videoconferenze programmate restano il canale sincrono; il repository GitHub del gruppo quello per codice e tracciatura delle attività.

Incontri in presenza. Non sono previsti incontri presso le sedi di Halue. Eventuali occasioni di incontro sono quelle istituzionali organizzate dall'Università nell'ambito del corso.

Supporto specialistico. Halue rende disponibili competenze su Shopify Plus e sull'architettura di integrazione, e mette a disposizione il proprio punto di vista operativo su come la documentazione viene realmente usata durante un passaggio di consegne — che è, in ultima analisi, il criterio con cui il prodotto verrà giudicato.

14. Uso dell'intelligenza artificiale nello sviluppo

L'uso di strumenti di intelligenza artificiale per lo sviluppo è **previsto, legittimo e incoraggiato**: assistenti di codice, generazione di test, refactoring assistito, esplorazione di API sconosciute. Halue non pone alcuna restrizione e non chiede che il contributo di questi strumenti venga dichiarato o contenuto.

Vale la pena dire perché, con franchezza. **La complessità medio-alta di questo capitolato è calibrata assumendo che quegli strumenti vengano usati.** Un capitolato tarato sul ritmo della scrittura manuale del codice sarebbe oggi tarato sulla variabile sbagliata: quello che era il collo di bottiglia non lo è più.

Ciò che cambia è **dove si sposta la difficoltà** — e quindi dove si sposta l'apprendimento. Se il codice si scrive in fretta, il valore del lavoro non sta più nell'averlo scritto: sta nelle decisioni che quel codice incarna e nella capacità di difenderle. Questo progetto è costruito attorno a decisioni che nessuno strumento può prendere al posto di chi progetta:

- **come modellare il grafo** delle entità e delle dipendenze, e quali relazioni valga la pena rappresentare;
- **dove tracciare il confine tra gli strati**, perché l'estensibilità richiesta da RNF-03 non si ottiene per caso;
- **quanto campionare** per affermare che un campo non è usato, e con quale margine di errore;
- **come comportarsi di fronte all'incertezza**: cosa dichiarare accertato, cosa probabile, cosa taciuto;
- **quali compromessi accettare** tra completezza della scansione, tempo di esecuzione e limiti dell'API.

Quello che Halue si aspetta di trovare in sede di verifica non è quindi la prova che il codice sia stato scritto a mano, ma:

1. **Comprensione dell'architettura, non solo del codice.** Ogni persona del gruppo deve saper spiegare come il sistema è fatto e perché.
2. **Consapevolezza delle alternative scartate.** È esattamente ciò che gli ADR richiesti al capitolo 10 servono a registrare: quali strade sono state considerate, perché sono state abbandonate, cosa si è accettato di perdere.
3. **Padronanza del dominio.** Il modello a punti delle Admin API, la semantica dei metafield, il funzionamento dei blocchi app non si delegano: vanno capiti.

4. **Responsabilità piena sul codice consegnato.** Non esiste codice di cui il gruppo non risponda perché "l'ha generato lo strumento". Se una riga entra nel repository, qualcuno la sostiene.
5. **Verifica indipendente.** I test sono il punto in cui l'assistenza automatica aiuta di più e serve di meno: generare casi è facile, decidere *cosa* vale la pena verificare è il lavoro vero.

Il criterio operativo che Halue applicherà è semplice: **ogni parte del sistema deve avere almeno una persona del gruppo in grado di spiegarla a voce, senza guardare il codice, e di argomentare perché è fatta così e non in un altro modo.** Dove questo non è vero, quella parte non è finita — indipendentemente da quanto funzioni.

15. Riferimenti aziendali

Ruolo	Nome	Contatto
Referente unico, product owner	**Valentino Baraldo** — Amministratore	valentino.baraldo@halue.io

Halue S.r.l. Società Benefit · hello@halue.io · halue.com

Sede legale: Viale Mentana 41, 43121 Parma (PR) — Sede operativa: Via A. Brustolon 14, 35031 Abano Terme (PD)

Appendice A — Glossario

Termine	Definizione	Approfondimento
ADR	*Architecture Decision Record*: nota breve che registra una decisione architetturale, le alternative valutate e le sue conseguenze	adr.github.io
Admin API	Interfaccia di amministrazione di Shopify, in versione GraphQL, che espone configurazioni e dati dello store	shopify.dev/docs/api/admin-graphql
App block / App embed	Componenti che un'app Shopify inserisce nel tema; i primi in posizioni definite dal template, i secondi come inserimenti globali. Il loro stato di abilitazione è registrato nella configurazione del tema	shopify.dev/docs/apps/build/online-store
B2B	*Business-to-business*: commercio rivolto ad altre imprese, con listini, cataloghi e condizioni dedicate	help.shopify.com/en/manual/b2b
B2C	*Business-to-consumer*: commercio rivolto al consumatore finale	—
Bulk operation	Meccanismo di Shopify per estrarre grandi volumi di dati in modo asincrono, aggirando i limiti delle query sincrone	shopify.dev/docs/api/usage/bulk-operations/queries
C4	Modello di documentazione architetturale a quattro livelli di zoom: contesto, contenitori, componenti, codice	c4model.com
CI	*Continuous Integration*: pratica di integrare e verificare automaticamente ogni modifica sul ramo principale	martinfowler.com/articles/continuousIntegration

Termine	Definizione	Approfondimento
CRM	*Customer Relationship Management*: sistema di gestione della relazione con il cliente	—
Custom app	App creata direttamente nell'amministrazione di un singolo store, non distribuita sull'App Store; è il modello di accesso previsto per Sonar	shopify.dev/docs/apps/build/authentication-authorization
Dark data	Dato presente e valorizzato nel sistema ma non letto da alcun consumatore noto	—
Durabilità (di un workflow)	Proprietà per cui l'esecuzione sopravvive a crash, riavvii e nuovi deploy, riprendendo dall'ultimo passo completato	vercel.com/docs/workflows/concepts
GraphQL	Linguaggio di interrogazione per API in cui il client dichiara esattamente i campi che vuole ricevere	graphql.org/learn
iPaaS	*Integration Platform as a Service*: piattaforma gestita per integrare sistemi eterogenei senza costruire l'infrastruttura di integrazione	gartner.com/en/information-technology/glossary
JSON-LD	*JSON for Linking Data*: formato JSON che rende esplicite le relazioni tra entità, adatto a scambiare grafi	json-ld.org
Liquid	Linguaggio di templating dei temi Shopify	shopify.dev/docs/api/liquid
Metafield	Campo personalizzato associabile a un'entità Shopify (prodotto, variante, ordine, cliente…), definito per `namespace` e `key`	shopify.dev/docs/apps/build/custom-data/metafields
Metaobject	Struttura dati personalizzata definita dall'utente, indipendente dalle entità native	shopify.dev/docs/apps/build/custom-data/metaobjects
MSW	*Mock Service Worker*: libreria che intercetta le chiamate di rete nei test, restituendo risposte registrate al posto del servizio reale	mswjs.io
MVP	*Minimum Viable Product*: versione minima ma completa e utilizzabile del prodotto	—
OAuth 2.0 / OIDC	Standard di autorizzazione delegata e, sopra di esso, di autenticazione dell'identità (*OpenID Connect*): permettono l'accesso con un provider esterno senza gestire password	oauth.net/2 · openid.net/developers/how-connect-works
Orfano	Entità che esiste nello store ma non ha alcun consumatore o riferimento noto	—
ORM	*Object-Relational Mapping*: strato che traduce tra le tabelle della base dati e gli oggetti del linguaggio di programmazione	—
PB	*Product Baseline*: consegna finale del progetto didattico, con prodotto e documentazione completi	—
PoC	*Proof of Concept*: prototipo che dimostra la fattibilità tecnica di un'ipotesi	—

Termine	Definizione	Approfondimento
Rate limiting	Limitazione della frequenza di chiamata imposta dall'API; Shopify adotta un modello a credito con recupero graduale	shopify.dev/docs/api/usage/rate-limits
Replatforming	Migrazione di uno store da una piattaforma e-commerce a un'altra (ad esempio da Salesforce Commerce Cloud a Shopify Plus), tipicamente motivata da limiti tecnici, di costo o di scalabilità della piattaforma di partenza	—
RLS	*Row Level Security*: meccanismo di PostgreSQL che impone l'isolamento dei dati a livello di riga direttamente nella base dati	supabase.com/docs/guides/database/postgres/row-level-security
RSC	*React Server Components*: componenti eseguiti sul server, che raggiungono il browser già renderizzati	react.dev/reference/rsc/server-components
RTB	*Requirements and Technology Baseline*: consegna intermedia del progetto didattico, con requisiti, architettura e prototipo	—
Script tag	Meccanismo legacy di Shopify per iniettare JavaScript di terze parti nelle pagine dello store	shopify.dev/docs/api/admin-graphql — `ScriptTag`
SDK	*Software Development Kit*: insieme di librerie e strumenti che una piattaforma fornisce per essere programmata	—
Shopify Function	Estensione che modifica la logica nativa di Shopify (sconti, spedizioni, pagamento) eseguita sull'infrastruttura Shopify	shopify.dev/docs/apps/build/functions
Snapshot	Fotografia immutabile e datata dello stato dello store prodotta da una scansione	—
Store enterprise	Negozio online di grandi dimensioni — molti prodotti, traffico consistente, più persone che vi hanno lavorato nel tempo — non una classificazione ufficiale di Shopify, ma il modo in cui questo capitolato descrive lo store bersaglio di Sonar	—
TLS	*Transport Layer Security*: protocollo che cifra e autentica le comunicazioni di rete; è ciò che sta sotto a HTTPS	datatracker.ietf.org/doc/html/rfc8446
UML	*Unified Modeling Language*: notazione standard per diagrammi di casi d'uso, classi, sequenza e attività	omg.org/spec/UML
UX / UI	*User Experience* e *User Interface*: rispettivamente l'esperienza complessiva d'uso e la superficie con cui l'utente interagisce	—
Webhook	Notifica HTTP che Shopify invia a un endpoint esterno al verificarsi di un evento	shopify.dev/docs/apps/build/webhooks

Appendice B — Riferimenti tecnici

Shopify

- Admin GraphQL API — https://shopify.dev/docs/api/admin-graphql
- `appInstallations` — https://shopify.dev/docs/api/admin-graphql/latest/queries/appInstallations
- Metafield e metaobject — https://shopify.dev/docs/apps/build/custom-data
- Webhook — https://shopify.dev/docs/apps/build/webhooks
- Temi e file di tema — https://shopify.dev/docs/storefronts/themes
- Shopify Functions — https://shopify.dev/docs/apps/build/functions
- Limiti di frequenza delle API — https://shopify.dev/docs/api/usage/rate-limits
- Bulk operations — https://shopify.dev/docs/api/usage/bulk-operations/queries

Vercel

- Vercel Workflows — https://vercel.com/docs/workflows
- Workflow SDK — https://workflow-sdk.dev
- Vercel Queues — https://vercel.com/docs/queues
- Cron Jobs — https://vercel.com/docs/cron-jobs
- Vercel Blob — https://vercel.com/docs/vercel-blob
- Vercel Sandbox — https://vercel.com/docs/vercel-sandbox
- AI Gateway — https://vercel.com/docs/ai-gateway
- AI SDK — https://ai-sdk.dev
- Next.js — https://nextjs.org
- Piano Hobby e limiti — https://vercel.com/pricing

Supabase

- Documentazione — https://supabase.com/docs
- Piano Free e limiti — https://supabase.com/pricing
- Row Level Security — https://supabase.com/docs/guides/database/postgres/row-level-security
- Auth con Google — https://supabase.com/docs/guides/auth/social-login/auth-google

Metodo e strumenti

- Modello C4 — https://c4model.com
- Mermaid — https://mermaid.js.org
- Architecture Decision Records — https://adr.github.io
- Vitest — https://vitest.dev · Playwright — https://playwright.dev · MSW — https://mswjs.io
- Drizzle ORM — https://orm.drizzle.team · shadcn/ui — https://ui.shadcn.com · React Flow — https://reactflow.dev

Documento rilasciato da Halue S.r.l. Società Benefit per il corso di Ingegneria del Software, Corso di Laurea in Informatica, Università degli Studi di Padova — A.A. 2026/2027.

© 2026 Halue S.r.l. Società Benefit — Il presente documento è classificato Public e può essere liberamente distribuito nell'ambito del corso.